Inside AgentOutput

Week 4, Lecture 2 · Browser Use: How LLM Agents Drive the Web

Summer 2026 · Every field, the message manager, and the LLM layer

What we'll cover

- The five fields of AgentOutput and what each one is for
- AgentBrain and the current_state property
- How the MessageManager assembles system prompt, history, and browser state
- Inspecting agent.history after a run
- The native LLM layer: ChatOpenAI, ChatAnthropic, ChatGoogle (not LangChain)

The five fields of AgentOutput

Field 1: evaluation_previous_goal

```
evaluation_previous_goal: str | None = None
```

The agent's own judgment of whether the prior step succeeded.

- **Step 1:** always "N/A" or similar (no prior step to evaluate)
- **Steps 2+:** the model reads the new browser state and compares it to what it aimed to do

Example: `"Success: the search results page loaded with MacBook Air listings"`

This field is procedural self-monitoring. If the agent writes "Success" on a step that clearly failed, you have found a reasoning error to investigate.

Field 2: memory

```
memory: str | None = None
```

The agent's running note to itself. Carries forward:

- The task it is working on
- Facts it has extracted (a title, a price, a username)
- Context it needs across steps that might fall out of the prompt window

Why this exists: the MessageManager trims older messages to fit the context budget. Memory is the mechanism for keeping critical facts in scope when history gets cut.

(browser-use contributors, 2026, agent/views.py)

Field 3: next_goal

```
next_goal: str | None = None
```

The agent's plain-English statement of what it intends to accomplish *this step*, written before selecting the action.

Reading `next_goal` and the `action` list together is the most direct audit:

- Does the action actually serve the stated goal?
- A mismatch between `next_goal` and `action` is a common signal that the agent is confused about page state

Field 4: action (the list)

```
action: list[ActionModel] = Field(..., json_schema_extra={"min_items": 1})
```

A list of one or more `ActionModel` instances. Each entry:

- Key: the action name (`click_element`, `go_to_url`, `input_text`, ...)
- Value: a dict of typed parameters specific to that action

For `click_element`, the parameter is `index`: the integer from the selector map.

```
{"click_element": {"index": 7}}
```

This is the link between the decision layer and the perception layer from week 2.

The agent can list multiple actions per step (multi-act): click, type, and scroll in one round.

Field 5: thinking (optional)

```
thinking: str | None = None
```

When `use_thinking=True` (default), the model writes a free-text reasoning block before committing to the structured reply. Not acted on by the framework; it is the model's scratchpad.

With `flash_mode=True` (faster, simpler mode), this field and `evaluation_previous_goal` are stripped from the schema entirely.

(browser-use team, 2026, agent-settings docs)

AgentBrain: a view, not a second output

```python
class AgentBrain(BaseModel):
    thinking: str | None = None
    evaluation_previous_goal: str
    memory: str
    next_goal: str
```

`AgentBrain` is **not** what the LLM produces separately. It is a named view onto the three cognitive-state fields, exposed via:

```python
brain = output.current_state  # returns AgentBrain
```

Internal code can pass `AgentBrain` around without carrying the full action list. The fields come from `AgentOutput`; nothing new is produced.

(browser-use contributors, 2026, agent/views.py)

The MessageManager

Three sources, one prompt

Each step, `MessageManager` assembles:

1. **System message** (from `SystemPrompt`): task, all action descriptions, constraints, AgentOutput format instructions
2. **Prior step history**: earlier steps as alternating user/assistant messages, trimmed to fit the context budget
3. **Current browser-state summary**: serialized DOM with element indices, URL, title, optional screenshot

The order matters. The system message sets the rules. History provides context. The current state gives the decision material.

What the system prompt contains

The system prompt is built from a `SystemPrompt` object. It includes:

- The task the agent is trying to complete
- Descriptions of every available action (from the tools registry)
- Instructions for how to fill each field of AgentOutput
- Any constraints: step limits, domain restrictions, format rules

You can extend it: `extend_system_message="Always prefer keyboard navigation."`
You can replace it: `override_system_message="..."` (use with care).

(browser-use team, 2026, agent-settings docs)

Context budget pressure

The context budget (week 3) governs all three message sources:

- If the system prompt is long (many tools, complex instructions), history gets fewer slots
- If history is long, the current browser-state summary may be trimmed
- `max_history_items` sets an explicit cap on stored steps

This is why `memory` exists: the agent writes facts it cannot afford to lose into that field, so they survive history trimming.

The native LLM layer

LangChain is no longer required

Until 2025, browser-use used LangChain as the integration layer. LangChain abstracts over many providers but adds:

- A dependency chain
- Version friction between LangChain and provider SDK releases
- An extra abstraction layer you have to debug through

The browser-use community migrated away from this. GitHub issue #2137 documents the switch. LangChain is no longer a required dependency.

(browser-use community, 2025, issue #2137)

The native wrappers: browser_use/llm

```python
from browser_use.llm import ChatOpenAI, ChatAnthropic, ChatGoogle

# GPT-4o
agent = Agent(task="...", llm=ChatOpenAI(model="gpt-4o"))

# Claude Sonnet
agent = Agent(task="...", llm=ChatAnthropic(model="claude-3-5-sonnet-20241022"))

# Gemini Flash
agent = Agent(task="...", llm=ChatGoogle(model="gemini-2.0-flash"))
```

Also available: `ChatGroq`, `ChatOllama`, `ChatDeepSeek`, `ChatMistral`, `ChatOpenRouter`, `ChatBrowserUse`, and more.

(browser-use team, 2026, supported-models docs)

What the wrappers handle

Each native wrapper:

- Speaks directly to the provider's API (no LangChain bridge)
- Handles authentication (reads the right environment variable)
- Implements function calling / structured output for that provider
- Manages streaming and retry logic

The rest of the framework sees a uniform interface. Swapping `ChatOpenAI` for `ChatAnthropic` changes nothing about AgentOutput, the selector map, or the tools registry.

> "LangChain is NOT required (it was removed as a core dependency; a community migration issue documents the switch). You pass a chat model into Agent(task=..., llm=...)."

Inspecting the full run

```
# Save the raw message history for every step
agent = Agent(
    task="...",
    llm=ChatOpenAI(model="gpt-4o"),
    save_conversation_path="./run.json",
)
result = await agent.run()

# Read the structured history
for item in agent.history.history:
    print(item.metadata.step_number, item.model_output.next_goal)
```

`save_conversation_path` writes the exact messages sent to the LLM, including the assembled system prompt and browser-state summary. The most direct debugging tool available.

Take-aways

1. `AgentOutput` has five fields: `evaluation_previous_goal`, `memory`, `next_goal`, `action` (list), and optional `thinking`. Together they are the complete record of one decision.
2. `AgentBrain` groups the three cognitive-state fields into a named view via `current_state`; it is not a separate LLM output.
3. `MessageManager` assembles system prompt, trimmed history, and browser-state summary into each LLM call; context budget governs what fits.
4. LangChain is not required. Import from `browser_use.llm` directly.
5. `agent.history` and `save_conversation_path` are the two main tools for inspecting what the model actually decided and why.

What's next

- **Section this week:** Capture real AgentOutput objects, read all fields, trace an element index back to the selector map from week 2
- **Week 5:** The tools registry: how `AgentOutput.action` becomes a real browser action
- **Week 5 reading:** ActionModel, ActionResult, multi-act dispatch, and injected dependencies